Lesson 01 · The .NET Platform Map

C# · .NET · VB for a Kotlin/Java, TypeScript & Python architect — runtime, languages, versions and toolchain

▤ Contents

1 · Why this track exists — and who it is written for 1
- 1.1 · The numbers to leave this lesson with 2

2 · The platform in one picture 2
- 2.1 · From source file to running process 3
- 2.2 · Asking the runtime what it is — Kotlin on the JVM vs C# 3
- 2.3 · Concept map — JVM and friends → .NET 3
- 2.4 · Runtime pieces — the JVM parts under their .NET names 4

3 · Versions — how to read a .NET codebase's age 5
- 3.1 · Support windows of modern .NET 5
- 3.2 · Support length by release 6
- 3.3 · Days of support left 6
- 3.4 · Target framework monikers you will find in project files 6
- 3.5 · What am I looking at? — dating a legacy repository 7
- 3.6 · A net10.0 app on a host that only has .NET 8 7

4 · Three languages, one runtime 8
- 4.1 · The shared C# library — a positional record 8
- 4.2 · The same program in C# and in Visual Basic 8
- 4.3 · What each language can do with the same C# record 9
- 4.4 · Language feature matrix — C#, VB and the languages you know 9

5 · The toolchain — one CLI, five publish options 9
- 5.1 · The dotnet CLI next to the tools you use today 10
- 5.2 · What happens on dotnet run (simplified) 10
- 5.3 · Two project files — the language is the file extension 11
- 5.4 · Five publish options — and how they combine 11

6 · Your transfer map and learning path 12
- 6.1 · Transfer map — how much of each lesson you already know 13
- 6.2 · Learning path 13
- 6.3 · Where this track's study time goes 14
- 6.4 · The lessons 14
- 6.5 · What carries over, what to learn, what to unlearn 14

7 · Hands-on — build and run the lesson-01 samples 14
- 7.1 · The two files every sample inherits 15
- 7.2 · Commands 16
- 7.3 · What you should see 16
- 7.4 · Lines of code in the lesson-01 samples 16

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why this track exists — and who it is written for

This track is written for one reader: an architect with eight years across Kotlin/Java, TypeScript, Python and Dart who has never had to own a .NET codebase — and keeps meeting one. Many insurance and banking estates run line-of-business systems on .NET, and the Windows (IIS) workloads that sit beside ECS Fargate and Lambda in a migration inventory are where .NET Framework most often hides. Reading that code, sizing its migration and designing its replacement is architecture work; this track gives you the vocabulary to do it first-hand.

The approach is map, don't restart. Every lesson starts from something you have already built — a Kotlin mono-repo of 50–120+ modules, a Spring WebFlux service, a Python Lambda tuned to ~175 ms cold starts, a 100%-coverage TDD suite, an Entra ID single sign-on integration — and shows the .NET shape of the same idea. Chips say whether an idea transfers as-is (● Same idea), only changes its name (● Renamed), genuinely behaves differently (● Different) or looks the same and bites (● Trap).

New .NET work is C#, but estates from the 2000s still run Visual Basic .NET, so you need to read both. A modernisation architect has to read VB, keep it running and plan its exit. VB appears beside C# wherever the difference teaches something, and lesson 06 (VB.NET for Legacy Estates) is dedicated to it.

Every code panel is real. C# and VB snippets are cut from sample projects in this repository by `#region` name, and `0-script/verify_samples.py` builds, tests and runs those projects on the .NET 10 SDK. A snippet that did not compile could not appear here.

1.1 · The numbers to leave this lesson with

.NET 10 · LTS	.NET 8 & 9 SUPPORT	LANGUAGES, ONE RUNTIME	THIS TRACK	LESSON 01 SAMPLES
to Nov 2028	**50 days**	**3**	**76 h**	**63 lines**
supported until 14 Nov 2028 · verified	end 10 Nov 2026 · counted from 2026-09-21	C# · Visual Basic · F# · verified	13 lessons · study-time estimate	C# + VB · measured at build

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured runtime tooling

i Before you start

1. **No earlier lesson is needed — this is where the track starts.** You will build `L01.PlatformKit`, a C# class library that reports the runtime a program is running on, and use it unchanged from a C# console app (`L01.CSharpApp`) and a Visual Basic console app (`L01.VbApp`).
2. **Install the .NET 10 SDK** — 10.0.401 or any later 10.0 SDK, which is what this repository's `global.json` accepts (download .NET 10). `dotnet --list-sdks` must list it. Python 3 runs `0-script/verify_samples.py`, which needs no extra packages.
3. **If `dotnet` says "No .NET SDKs were found", a runtime-only install earlier on PATH is shadowing the SDK.** Put the SDK's `dotnet` folder first on PATH, or call that `dotnet.exe` by its full path. `DOTNET_ROOT` does not help: it tells an apphost where the runtime is, not which SDKs the `dotnet` command sees. Visual Studio installs SDKs of its own too (lesson 13 (Working in Visual Studio), section 3).
4. **Every numbered section leads with its point.** Evidence, code and diagrams follow, so the first paragraph of each section gives you the whole argument.
5. Facts that change with releases are marked • Verified and link to the official source; judgements such as study hours are marked • Estimate; numbers computed from this repository at build time are marked • Measured. Code panels name the file and `#region` they were cut from; Java, Kotlin, TypeScript and Python panels are for comparison and are not compiled.

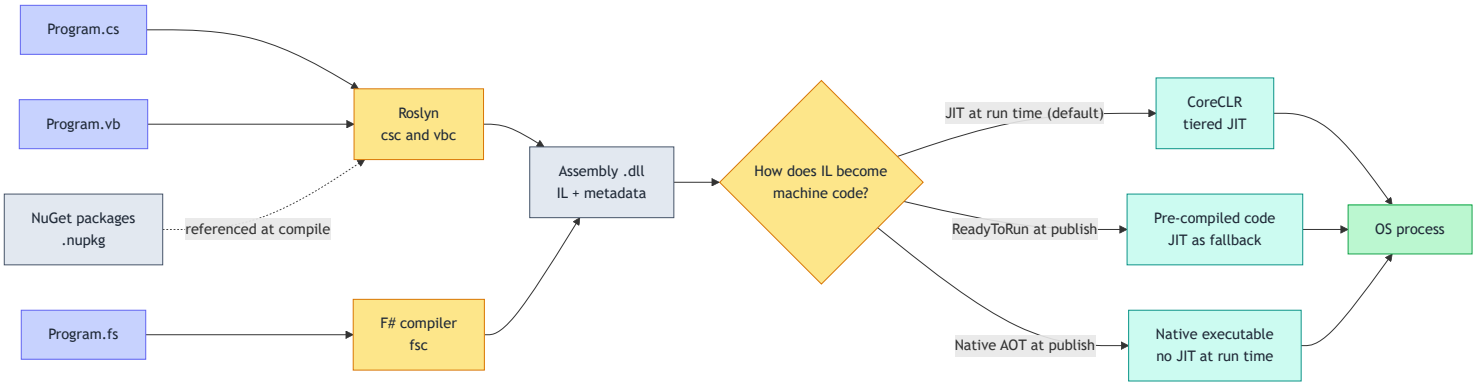
2 · The platform in one picture

.NET is a runtime plus a class library plus compilers — the same three-part shape as the JDK, with one decisive difference: the runtime was designed for many languages from day one. C#, Visual Basic and F# compile to the same intermediate language (IL), load into the same runtime and share the same type system. A VB program can call a C# library with no bridge, adapter or marshalling layer — the lesson-01 samples do exactly that.

The pieces map almost one-to-one onto the JVM. CoreCLR is HotSpot. IL inside an assembly is bytecode inside a JAR. The Base Class Library is the JDK class library. NuGet is Maven Central. Roslyn — the C# and VB compilers, written in C# and VB themselves — is javac with a public API that analyzers and IDEs call directly.

Four things stop mapping: generics, value types, exceptions and how IL becomes machine code. Generics are *reified*: `List<int>` is a real type at run time and stores unboxed integers. User-defined value types (`struct`) are stored inline in whatever holds them — a local, an array, an object's field — instead of as separate heap objects, and they copy on assignment. Exceptions are all unchecked. And the compiled output does not have to stay IL: ReadyToRun pre-compiles it at publish time and Native AOT removes the JIT altogether — GraalVM native-image is the nearest thing you know.

2.1 · From source file to running process



indigo = source · amber = compiler or decision · slate = artefact · teal = runtime path · green = the OS process · framework-dependent vs self-contained is a separate switch (5.4)

2.2 · Asking the runtime what it is — Kotlin on the JVM vs C#

Kotlin — for comparison the JVM questions you already ask

```kotlin
// no single JVM API holds all of this: the nearest calls
fun capture(caller: Class<*>) = PlatformReport(
    framework = "Java ${Runtime.version()}",
    // no stored target framework: a .class file records
    // only its format version (major 65 = Java 21)
    operatingSystem = System.getProperty("os.name"),
    arch = System.getProperty("os.arch"),
    gc = ManagementFactory.getGarbageCollectorMXBeans()
        .joinToString { it.name },
    // null when the JVM has no JIT compiler
    jitOn = ManagementFactory.getCompilationMXBean() != null,
    callerJar = caller.protectionDomain.codeSource
        ?.location?.path ?: "(unknown)",
)
```

C# samples/L01.PlatformKit/PlatformReport.cs · #region capture

```csharp
public static PlatformReport Capture(Assembly caller) => new(
    Framework: RuntimeInformation.FrameworkDescription,
    TargetFramework: caller
        .GetCustomAttribute<TargetFrameworkAttribute>()
        ?.FrameworkName ?? "(none)",
    OperatingSystem: RuntimeInformation.OSDescription,
    Arch: RuntimeInformation.ProcessArchitecture,
    ServerGc: GCSettings.IsServerGC,
    JitOn: RuntimeFeature.IsDynamicCodeCompiled,
    ImageRuntimeVersion: caller.ImageRuntimeVersion,
    CallerAssembly: caller.GetName().Name ?? "(unnamed)");
```

.NET answers each question with one typed API; the JVM spreads them over system properties and management beans. `FrameworkDescription` is the runtime executing the process, after roll-forward; `TargetFrameworkAttribute` is what the assembly was built for — the JVM keeps no such record, only a class-file version. `IsDynamicCodeCompiled` is false under Native AOT, much as the compilation bean is null on a JVM without a JIT. The record this fills is shown in 4.1.

2.3 · Concept map — JVM and friends → .NET

You already know	In .NET	Kind	Watch for
JVM (HotSpot)	CoreCLR	• Renamed	Multi-language by design — not a VM for one language
Bytecode in .class	IL in an assembly (.dll)	• Renamed	An assembly is closer to a JAR than to a .class
JAR / fat JAR	Assembly / self-contained publish	• Renamed	The .exe apphost is a native launcher over IL, not a native build
A Java 8 JAR usually runs on a Java 21 JRE	A `net8.0` app needs an 8.0 runtime	• Trap	The host rolls forward to a newer patch or minor, never a newer major — unless `RollForward` is `Major`
JDK class library	Base Class Library (BCL)	• Same idea	Namespaces start with `System.`
Maven Central / npm	NuGet (nuget.org)	• Renamed	No separate pom or package.json — it is all in the project file
pom.xml / build.gradle.kts	.csproj / .vbproj (MSBuild)	• Different	Declarative XML; shared logic goes in Directory.Build.props — lesson 07 (Solutions, MSBuild & Mono-repos)
Generics with type erasure	Reified generics	• Different	`typeof(T)` works; `List<int>` stores unboxed integers
Java primitives (int, double) · Kotlin value class	User-defined value types (`struct`)	• Trap	Copies on assignment, so changing a copy changes nothing; casting to `object` or an interface boxes it
Checked exceptions	Unchecked exceptions only	• Different	No `throws`; document with `/// <exception>`
Kotlin data class / Java record	C# `record`	• Same idea	`with = copy()`; VB can use records but not declare them — lesson 03 (Types, OOP & Generics)

You already know	In .NET	Kind	Watch for
Kotlin coroutines / WebFlux	`async/await` + `Task`	• Different	lesson 05 (Async, Tasks & Concurrency)
GraalVM native-image	Native AOT	• Same idea	Same closed-world trade-off: reflection needs annotations
SDKMAN / .sdkmanrc / pyenv	global.json	• Renamed	Pins the SDK, not the runtime; it selects among installed SDKs and never installs one
Spring Boot	ASP.NET Core	• Different	Built-in container with explicit registration — no component scanning — lesson 08 (ASP.NET Core Web APIs)
Java EE app on WebSphere	ASP.NET on IIS (.NET Framework)	• Same idea	`System.Web` does not exist on modern .NET — port, don't lift
JUnit + Mockito	xUnit / NUnit / MSTest + NSubstitute / Moq	• Renamed	lesson 10 (Testing, TDD & 100% Coverage)

2.4 · Runtime pieces — the JVM parts under their .NET names

runtime view

1 CoreCLR — the Common Language Runtime

runtime • Renamed

Loads assemblies, verifies and JIT-compiles IL, runs the garbage collector, owns threads and exceptions.

JVM HotSpot / OpenJ9

DIFFERS One runtime serves every .NET language — C#, VB and F# share types directly

ROLL-FWD An app starts on a newer patch or minor of the runtime it targets — never on a newer major by default

OLD NAME The .NET Framework CLR is a separate, Windows-only runtime — see §3

2 IL, metadata and assemblies

runtime • Renamed

The compiler emits intermediate language plus rich type metadata into an assembly.

JVM Bytecode in .class files, packaged into a JAR

.NET IL + metadata in one assembly (.dll) that holds many types

TARGET The build stamps `TargetFrameworkAttribute` into it — `.NETCoreApp,Version=v10.0` in the samples

INSPECT ILSpy or ildasm — the javap of .NET

TRAP The .exe beside your .dll is a native *apphost* launcher; your program is still IL

3 Tiered JIT, dynamic PGO and ReadyToRun

runtime • Same idea

Start fast with a quick JIT, then recompile hot methods with full optimisation.

JVM C1 → C2 tiered compilation

.NET Tier-0 quick JIT → tier-1 optimised JIT; on by default since .NET Core 3.0

PGO Dynamic profile-guided optimisation is on by default since .NET 8

R2R ReadyToRun ships pre-compiled native code inside the assembly to cut start-up; JIT still available

2.4 · Runtime pieces (continued)

runtime view

4 Garbage collector and value types

runtime · • Different

A generational, compacting GC — plus a type system that avoids many allocations in the first place.

JVM	G1 / ZGC / Parallel collectors
MODES	Workstation or Server GC; ASP.NET Core apps default to Server GC
STRUCTS	Value types live inline (stack or inside an object) and copy on assignment
SAMPLE	Both lesson-01 apps print `server GC False` — the console default

5 Base Class Library and NuGet

tooling · • Renamed

A large standard library, and one public package registry.

JDK	java.base, java.net.http, java.time …
.NET	System.*, System.Net.Http, System.Text.Json, System.Collections.Generic
REGISTRY	nuget.org ≈ Maven Central; dependencies are PackageReference items in the project file
LOCK FILE	packages.lock.json is opt-in (RestorePackagesWithLockFile) — unlike npm

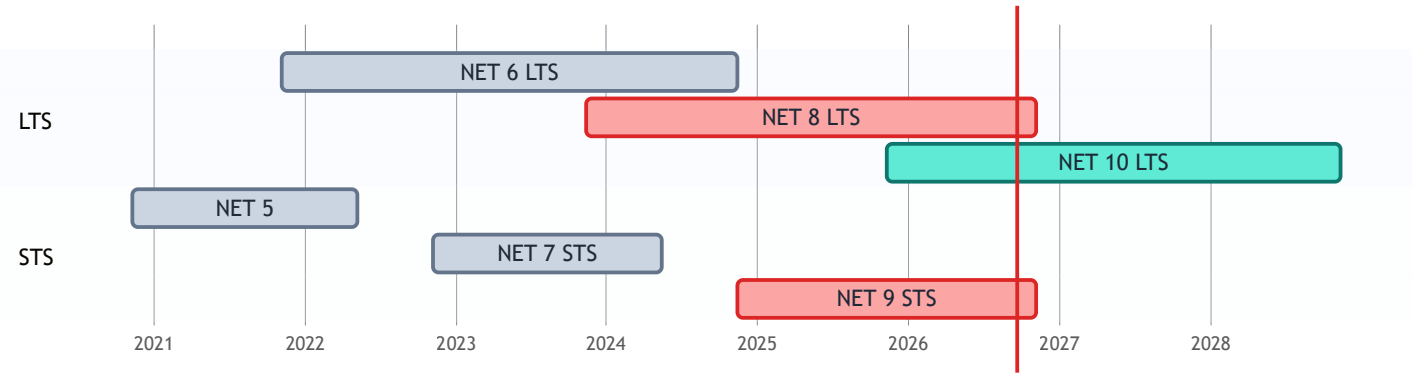
3 · Versions — how to read a .NET codebase's age

There are two .NETs, and a codebase's project file tells you which one it needs in the first ten lines. *.NET Framework* (1.0 in 2002 → 4.8.1 in 2022) is Windows-only, ships as a Windows component and receives no new major versions. *.NET* — born as .NET Core 1.0 in 2016 and renamed at .NET 5 in 2020, skipping "4" to avoid confusion — is cross-platform, open source and ships every November (releases and support).

The release rhythm is predictable. Even-numbered releases are Long Term Support (36 months); odd-numbered releases are Standard Term Support, extended from 18 to 24 months starting with .NET 9. That extension makes .NET 8 (LTS) and .NET 9 (STS) leave support on the **same day, 10 November 2026**. .NET 10 shipped on 11 November 2025 and is supported until 14 November 2028 (.NET support policy · .NET 8 and 9 end of support).

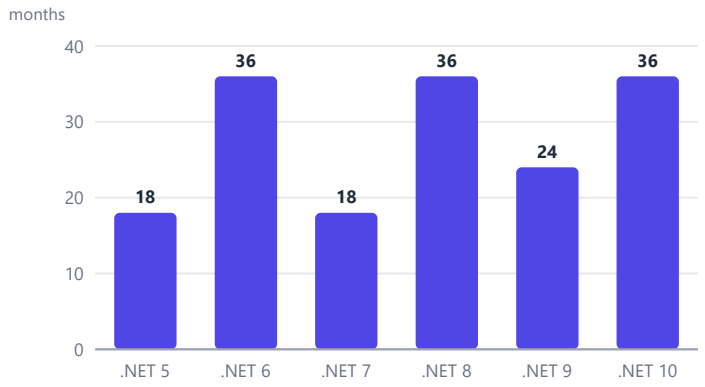
New work targets `net10.0`, and anything on `net8.0` or `net9.0` has 50 days left to get there. Anything on .NET Core 3.1, .NET 5, 6 or 7 is already unsupported. A .NET Framework 4.8.1 application is still supported as a component of the Windows version it runs on (.NET Framework lifecycle), but it will never get a new runtime feature — a migration argument rather than a security one. Retargeting is not optional: an app does not move to a newer major runtime on its own (3.6).

3.1 · Support windows of modern .NET



grey = out of support · red = ends 10 Nov 2026 · teal = current LTS · red vertical line = the day this PDF was built

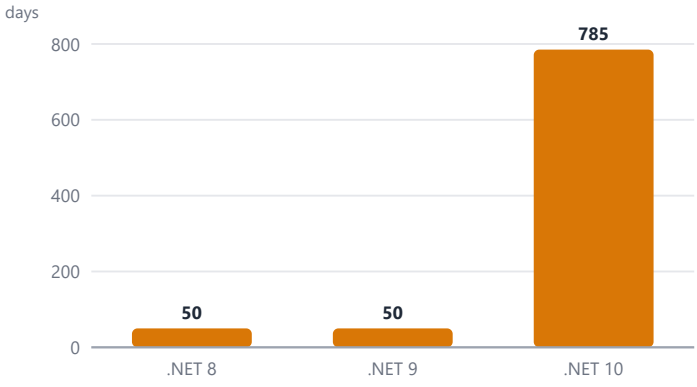
3.2 · Support length by release



months from GA to end of support · verified against the .NET support policy

Even = 36 months, odd = 18 → 24. The STS extension is why .NET 9 does not buy you any time over .NET 8: both end on 10 Nov 2026. • Verified .NET support policy.

3.3 · Days of support left



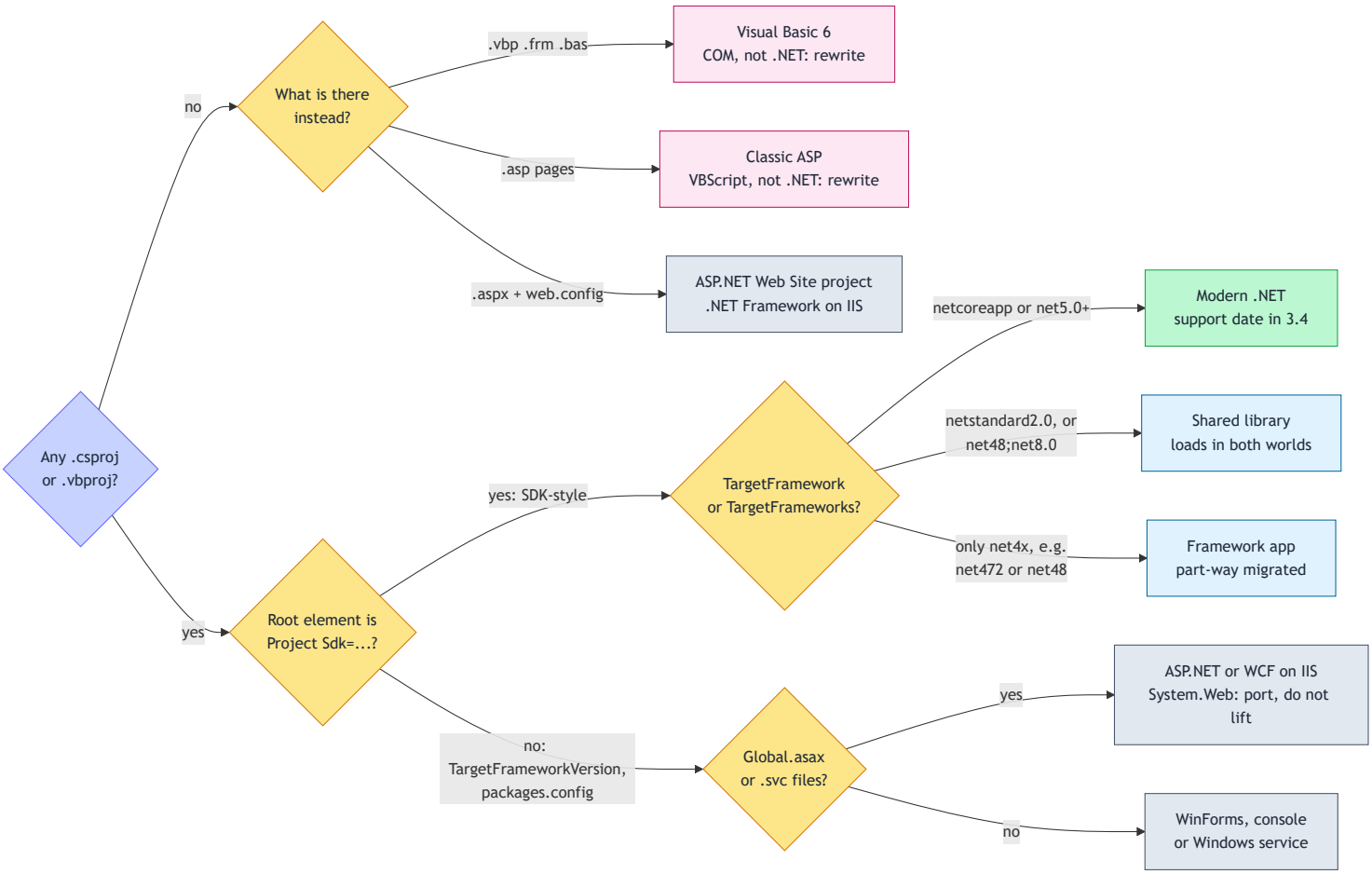
days from the build date 2026-09-21 to end of support · counted from the verified dates

.NET 8 and .NET 9 share one countdown — 50 days — while .NET 10 has 785. The bars are recounted on every build, so the first two drop to zero after 10 Nov 2026.

3.4 · Target framework monikers you will find in project files

TFM	What it targets	Where you meet it	Status
`<TargetFrameworkVersion>v4.x` (no Sdk attribute)	.NET Framework 4.x in an old-style project — Windows only	Unmigrated apps, often with `packages.config` and `Global.asax`	• 4.6.1 and older ended Apr 2022 • 4.6.2 ends 12 Jan 2027
`net48` · `net481`	.NET Framework 4.8 / 4.8.1 in an SDK-style project	Apps part-way through a migration; ASP.NET, WCF and WinForms on Windows servers	• Legacy · follows Windows
`netstandard2.0`	An API contract, not a runtime	Shared libraries that must load in both .NET Framework and modern .NET	• Bridge
`netcoreapp3.1`	.NET Core 3.1	First-wave migrations	• Ended Dec 2022
`net6.0` · `net7.0`	.NET 6 LTS · .NET 7 STS	Services built 2021–2023	• Ended 2024
`net8.0` · `net9.0`	.NET 8 LTS · .NET 9 STS	Most services running today	• Ends 10 Nov 2026
`net10.0`	.NET 10 LTS	New work — every sample in this track	• Supported to Nov 2028
`net10.0-windows`	.NET 10 plus Windows-only APIs	Modernised WinForms / WPF desktop apps	• Windows only

3.5 · What am I looking at? — dating a legacy repository



indigo = start · amber = question · green = modern · sky = bridge · slate = .NET Framework · rose = not .NET at all

3.6 · A net10.0 app on a host that only has .NET 8

Terminal — the net10.0 sample on an 8.0-only host captured on the build machine

```
# a host whose only runtimes are 8.0.19 and 8.0.28
dotnet L01.CSharpApp.dll
You must install or update .NET to run this application.

Architecture: x64
Framework: 'Microsoft.NETCore.App', version '10.0.0' (x64)
.NET location: C:\Program Files\dotnet\

The following frameworks were found:
  8.0.19 at [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
  8.0.28 at [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]

# exit code 150 · the App: path line and the download links are left out
```

> **The host rolls forward, never back — and never across a major version by default.** `L01.CSharpApp.runtimeconfig.json` asks for Microsoft.NETCore.App 10.0.0, so the host accepts any installed 10.0 patch but not 8.0. The reverse fails the same way: a `net8.0` app does not start on a host that has only .NET 10 unless its `RollForward` is `Major` (Select which .NET version to use).

⚠ The 10 November 2026 double cut-off

1. **.NET 8 (LTS) and .NET 9 (STS) stop receiving security fixes on the same day.** Upgrading from 8 to 9 buys nothing; the only supported destination is .NET 10.
2. **Start the inventory with the TFM, not the language.** A VB.NET service on `net8.0` is a one-line retarget plus a test run; a C# service on `net48` is a port.
3. **Installing the .NET 10 runtime upgrades nothing.** A `net8.0` app still needs an 8.0 runtime, because the default `RollForward` policy (`Minor`) never crosses a major version. Retarget and retest; set `RollForward` to `Major` only as a documented stop-gap (Select which .NET version to use).

4. **Change the TFM and the container base image together.** An app retargeted to `net10.0` whose Dockerfile still says FROM an 8.0 runtime image fails at start-up, as 3.6 shows; an app still on `net8.0` placed on a 10.0 image does not start either.
5. Sources: .NET 8 and 9 end of support · .NET support policy.

4 · Three languages, one runtime

C# is where .NET evolves; Visual Basic is deliberately stable; F# is the functional option. C# 14 ships with .NET 10 and keeps adding syntax — extension members, the `field` keyword, null-conditional assignment (What's new in C# 14). Microsoft's stated strategy for Visual Basic is *consumption-only*: VB code can use new .NET APIs and types, but the language will generally not add syntax to *define* things that need new language support (Visual Basic language strategy · .NET managed languages strategy).

A VB project uses a C# library as if it were written in VB — it just cannot declare everything C# can. Same types, same exceptions, same debugger. VB can read a C# `record`, compare it and pass it on; it cannot declare a record or use `with` to copy one. The samples below are the proof: one C# class library, consumed unchanged by a C# console app and a VB console app.

Mixed-language solutions need no FFI, so a VB-to-C# migration can move one project at a time. New code goes into C# libraries, old VB calls it, and each VB project is converted when it is next touched. lesson 06 (VB.NET for Legacy Estates) turns this into a plan.

4.1 · The shared C# library — a positional record

C# — samples/L01.PlatformKit/PlatformReport.cs · #region report-record

```
/// <summary>An immutable snapshot of the runtime a program is executing on.</summary>
public sealed record PlatformReport(
    string Framework,           // the runtime running this process: ".NET 10.0.12"
    string TargetFramework,     // what the assembly was BUILT for: ".NETCoreApp,Version=v10.0"
    string OperatingSystem,
    Architecture Arch,
    bool ServerGc,              // workstation (False) or server (True) garbage collector
    bool JitOn,                 // False under Native AOT: no code is compiled at run time
    string ImageRuntimeVersion, // "v4.0.30319" on every modern .NET: a CLR 4 header string
    string CallerAssembly);
```

One declaration gives you a constructor, init-only properties (settable only while the object is created, including by a `with` copy), value equality, `ToString()` and `Deconstruct` — Kotlin's `data class` in C#. Read the two version fields apart: `TargetFramework` is what the assembly was built for; `ImageRuntimeVersion` is the CLR version string written into the metadata header, still `v4.0.30319` — not the .NET version (Assembly.ImageRuntimeVersion).

4.2 · The same program in C# and in Visual Basic

C# — samples/L01.CSharpApp/Program.cs · #region program

```
var r = PlatformProbe.Capture(
    Assembly.GetExecutingAssembly());

Console.WriteLine($"Hello from C# on {r.Framework}");
Console.WriteLine($"  target    {r.TargetFramework}");
Console.WriteLine($"  os        {r.OperatingSystem}");
Console.WriteLine($"  arch      {r.Arch}");
Console.WriteLine($"  server GC {r.ServerGc}");
Console.WriteLine($"  JIT on    {r.JitOn}");
Console.WriteLine($"  image     {r.ImageRuntimeVersion}");
Console.WriteLine($"  assembly  {r.CallerAssembly}");
```

Visual Basic — samples/L01.VbApp/Program.vb · #region program

```
Dim r = PlatformProbe.Capture(
    Assembly.GetExecutingAssembly())

Console.WriteLine($"Hello from VB on {r.Framework}")
Console.WriteLine($"  target    {r.TargetFramework}")
Console.WriteLine($"  os        {r.OperatingSystem}")
Console.WriteLine($"  arch      {r.Arch}")
Console.WriteLine($"  server GC {r.ServerGc}")
Console.WriteLine($"  JIT on    {r.JitOn}")
Console.WriteLine($"  image     {r.ImageRuntimeVersion}")
Console.WriteLine($"  assembly  {r.CallerAssembly}")
```

Same library, same types, same output. C# uses top-level statements; VB needs a `Module` with `Sub Main` around this region. VB is case-insensitive and line-oriented — the `Capture(` call continues on the next line without a `_` because it ends with an open parenthesis.

4.3 · What each language can do with the same C# record

C# samples/L01.CSharpApp/Program.cs · #region equality

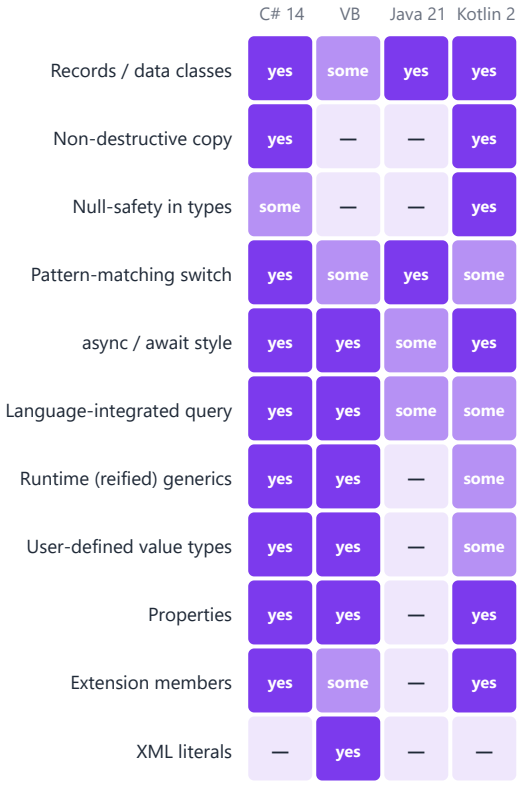
```csharp
// records compare by VALUE, like a Kotlin data class
var flipped = r with { ServerGc = !r.ServerGc };
Console.WriteLine($"  same?     {r == flipped}"); // False
Console.WriteLine($"  copy?     {r == r with { }}"); // True
```

Visual Basic samples/L01.VbApp/Program.vb · #region equality

```vb
' VB can USE the C# record (read it, compare it) but has
' no expression that makes a modified copy of it.
Dim same = r.Equals(PlatformProbe.Capture(
    Assembly.GetExecutingAssembly()))
Console.WriteLine($"  equal?    {same}")    ' True
```

C# can compare and copy; VB can only compare. Both get value equality from the record, so a fresh capture equals the first one. C# makes a modified copy with `with`; VB has no syntax for that — consumption-only in one screen. For a migration this is good news: shared models can move to C# first without breaking their VB callers. Records, `with` and equality in depth: lesson 03 (Types, OOP & Generics).

4.4 · Language feature matrix — C#, VB and the languages you know



yes = first-class syntax · some = library, subset, compile-time-only or consume-only · — = none · a rubric, not a benchmark

VB stopped gaining new syntax around 2017; its gaps are the features C# has added since. VB 15 took tuples alongside C# 7 but not C# 7's pattern matching, and later VB releases mostly learned to consume new C# constructs, such as init-only properties in VB 16.9 (What's new in Visual Basic). The gaps: pattern matching (C# 7, 2017), nullable reference types (C# 8, 2019), records and `with` (C# 9, 2020) (The history of C#); VB never had C#'s unsafe code either. Its one unique row — XML literals — is also why some legacy VB code is hard to port mechanically. Java's "some" async is virtual threads and `CompletableFuture`; Kotlin's "some" generics are `inline reified` functions. C#'s "some" in the null-safety row is deliberate: nullable reference types are compiler annotations, not runtime types (lesson 02 (C# Language Essentials)).

5 · The toolchain — one CLI, five publish options

The `dotnet` CLI does the jobs of Gradle and npm in one executable, and `global.json` does `.sdkmanrc`'s job of pinning the SDK. The CLI scaffolds from templates, restores packages, builds, tests, runs, packs and publishes. It selects among the SDKs installed on the machine but does not install one. This repository's `global.json` asks for 10.0.401 with `rollForward: latestFeature`, so the CLI uses the highest installed 10.0 SDK at or above that version (global.json overview). Restore is implicit in build, test and run, so the everyday loop is two commands.

A project file is declarative MSBuild, not a script. An SDK-style `.csproj` or `.vbproj` is often five lines, because the SDK supplies defaults: every source file in the folder is compiled, packages restore from nuget.org, output goes to `bin/`. Shared settings live in `Directory.Build.props`, which MSBuild imports for every project beneath it — this repository uses one to put all build output in `.build/artifacts`. lesson 07 (Solutions, MSBuild & Mono-repos) goes deep.

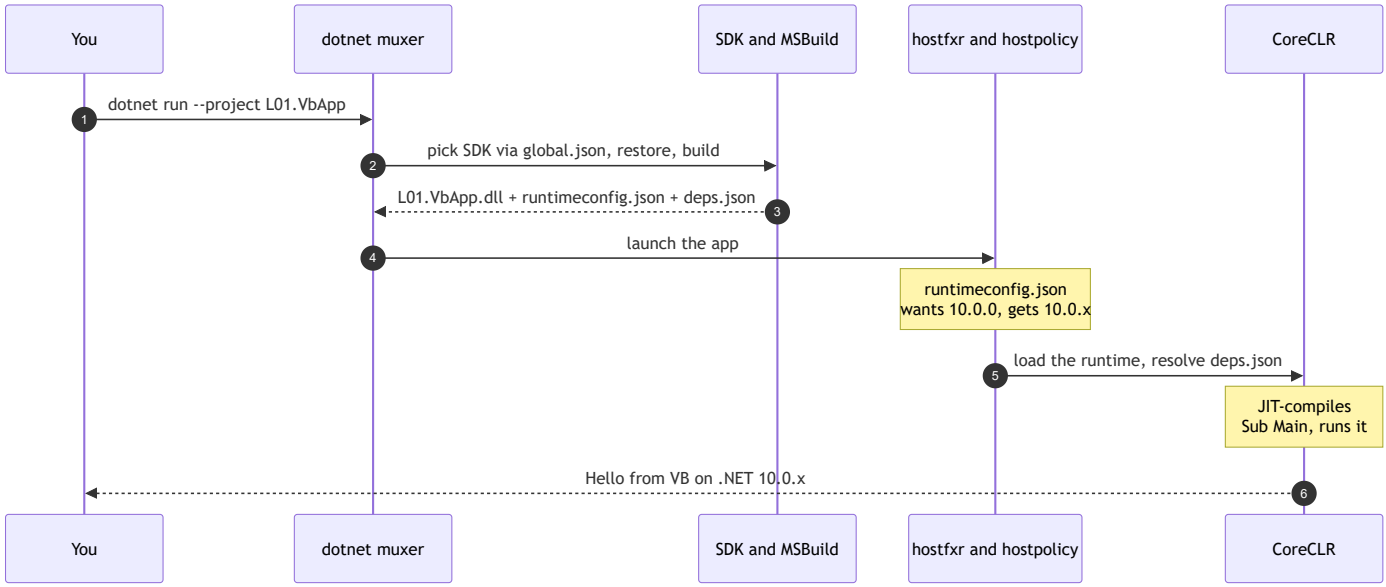
Publishing is a set of switches, not one choice. Framework-dependent or self-contained is one switch; single-file and ReadyToRun are independent switches that work with either; Native AOT implies self-contained. Together they trade start-up

time and package size against compatibility with reflection-heavy libraries — the same trade you made choosing between a JAR on a JRE and a GraalVM native image, and the one you tuned when you got Python Lambda packages down to 55 KB.

5.1 · The dotnet CLI next to the tools you use today

Task	Gradle · npm	.NET CLI	Note
Scaffold	`gradle init` · `npm init`	`dotnet new console -lang VB`	`dotnet new list` shows templates
Restore	implicit · `npm install`	`dotnet restore`	implicit in build, test and run
Compile	`gradle build` · `npm run build`	`dotnet build -c Release`	Debug is the default configuration
Test	`gradle test` · `npm test`	`dotnet test`	xUnit, NUnit and MSTest — lesson 10 (Testing, TDD & 100% Coverage)
Run	`gradle bootRun` · `npm start`	`dotnet run`	.NET 10 also runs one file: `dotnet run app.cs`
Package an app	`gradle bootJar`	`dotnet publish`	`-r linux-x64 --self-contained` · `-p:PublishAot=true`
Package a library	`gradle publish` · `npm publish`	`dotnet pack` · `dotnet nuget push`	produces a .nupkg
Add a dependency	edit the build file · `npm install x`	`dotnet package add X`	the .NET 10 noun-first form; `dotnet add package X` still works (What's new in the .NET 10 SDK)
Tools	SDKMAN · `npm i -g` · `npx`	`dotnet tool install -g` · `dnx`	dnx runs a tool once without installing it, like npx; repo-local tools live in `dotnet-tools.json`
Format / lint	Spotless, ktlint · Prettier, ESLint	`dotnet format`	rules come from `.editorconfig`

5.2 · What happens on dotnet run (simplified)



the muxer finds the SDK through global.json; the host finds the runtime through runtimeconfig.json, within one major version

5.3 · Two project files — the language is the file extension

MSBuild / XML samples/L01.CSharpApp/L01.CSharpApp.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference
      Include="..\L01.PlatformKit\L01.PlatformKit.csproj" />
  </ItemGroup>
</Project>
```

MSBuild / XML samples/L01.VbApp/L01.VbApp.vbproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <RootNamespace>L01.VbApp</RootNamespace>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference
      Include="..\L01.PlatformKit\L01.PlatformKit.csproj" />
  </ItemGroup>
</Project>
```

The same MSBuild, the same SDK. Neither file lists a source file or a target framework: the SDK compiles every `.cs` or `.vb` in the folder, and `net10.0` comes from the repository's `Directory.Build.props`. The VB app references the C# library exactly the way the C# app does.

5.4 · Five publish options — and how they combine

deployment view

6 Framework-dependent

tooling · Same idea

Ship your IL and dependencies; the machine supplies the .NET runtime.

LIKE	A thin JAR on an installed JRE · a Python zip on a Lambda runtime
SIZE	Smallest package — only your IL and its dependencies
PICK WHEN	AWS Lambda managed runtime, containers on the aspnet image, servers you patch
COMBINES	With single-file and ReadyToRun; the host still needs a matching major runtime

7 Self-contained

tooling · Renamed

Ship the runtime with the app, for one runtime identifier such as linux-x64.

LIKE	A jlink runtime image
COST	Every app carries its own runtime copy; runtime patches arrive only when you redeploy
PICK WHEN	Hosts where you cannot install or control the runtime
COMBINES	With single-file, ReadyToRun and trimming

8 Single-file

tooling · Renamed

Bundle the app (and optionally the runtime) into one executable file.

LIKE	A fat JAR with a native launcher
NOTE	Still JIT-compiled at start-up unless combined with ReadyToRun
PICK WHEN	CLI tools and agents you copy onto machines
COMBINES	Framework-dependent or self-contained

5.4 · Five publish options (continued)

deployment view

9 ReadyToRun `runtime` • Different

Pre-compile IL to native code at publish time and keep the JIT as a fallback.

LIKE	An ahead-of-time compilation cache — faster start, same compatibility
COST	Larger assemblies; hot methods are still re-compiled at tier 1
PICK WHEN	Start-up matters but you depend on reflection-heavy libraries
COMBINES	Framework-dependent or self-contained, single-file or not

10 Native AOT `runtime` • Same idea

Compile the whole app to a native executable — no JIT and no IL at run time.

LIKE	GraalVM native-image — the same closed-world trade-off
FOR YOU	The .NET answer to the ~175 ms cold starts you tuned in Python Lambdas — lesson 12 (Cloud-Native .NET on AWS & Modernization)
LIMITS	No run-time code generation; reflection needs trimming annotations
IMPLIES	Self-contained and trimmed; the sample's `JIT` on line would print False

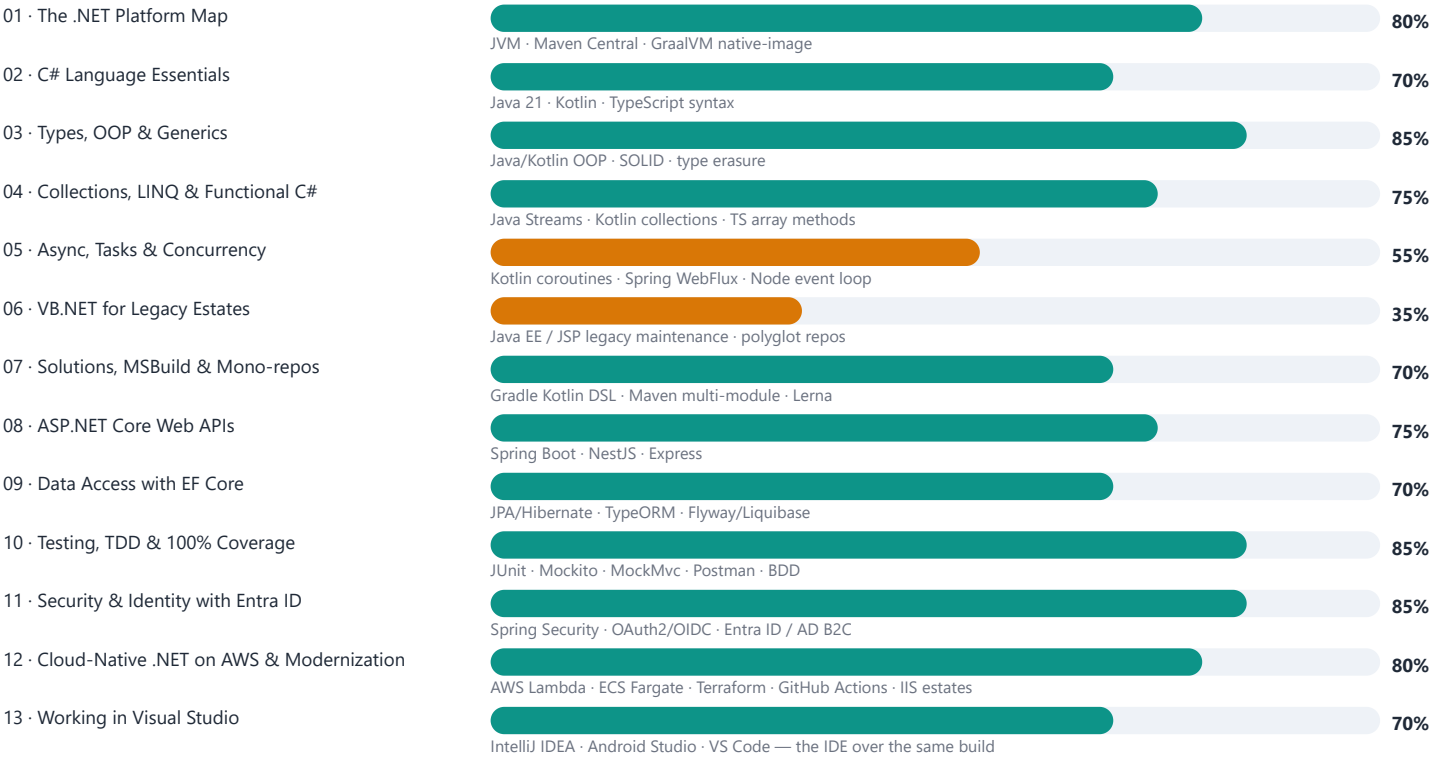
6 · Your transfer map and learning path

Most of this track is translation, and the chart below says where it is not. The estimate for each lesson is the share of its content that a Java/Kotlin, TypeScript and Python engineer already knows under another name. Two lessons fall well below the rest: asynchronous code, where C#'s `Task` model differs from both coroutines and reactive streams, and VB.NET, where the syntax maps word for word but the semantics quietly change numbers (lesson 06 (VB.NET for Legacy Estates)).

The path has five phases. *Foundations* makes you fluent in the language. *Runtime & codebase* covers concurrency, legacy VB and how large solutions are built — the three things that decide whether you can read an existing estate. *Services* builds, persists and tests an ASP.NET Core API. *Production* secures it with Entra ID and ships it to AWS, and closes with modernising a Windows / IIS application. *Tooling* is a companion: lesson 13 (Working in Visual Studio) covers Visual Studio, and you read it after lesson 07 or whenever your team lives in the IDE.

One running example ties the samples together. From lesson 02 onward the samples model a motor-insurance quoting service — vehicles, drivers, quotes, premiums and policies — so each lesson adds a layer to something you already understand rather than starting a new toy.

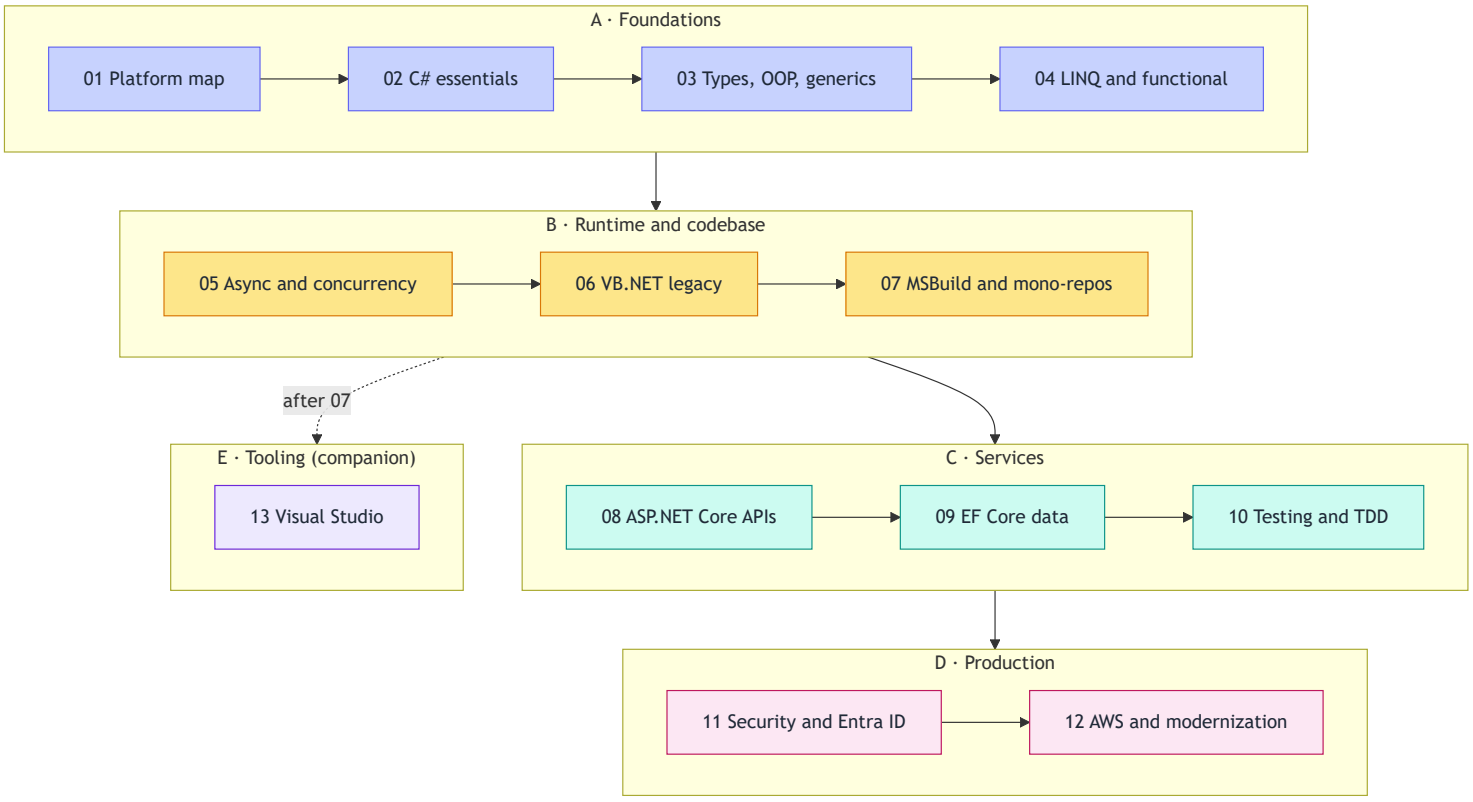
6.1 · Transfer map — how much of each lesson you already know



estimate · red below 34% · amber below 67% · the grey line under each bar names what it maps from

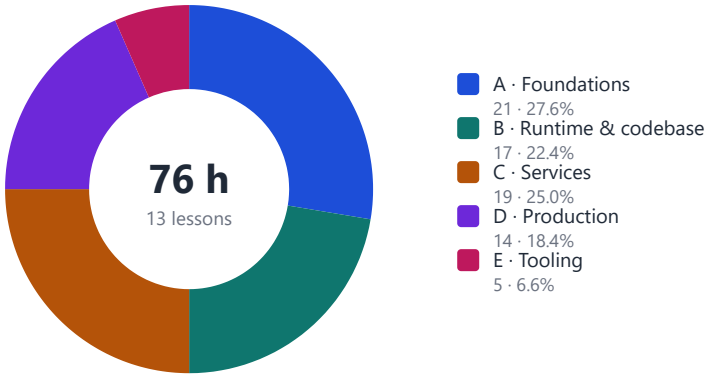
> **Spend the saved time on lessons 05 and 06.** Everything above 70% is mostly vocabulary; async (55%) and VB.NET (35%) are where a senior engineer from another stack most often writes code that compiles and is still wrong.

6.2 · Learning path



arrows = recommended order · dotted arrow = optional companion · colours = the five phases

6.3 · Where this track's study time goes



suggested hours per learning-path phase · estimate

> **Foundations take the largest share on purpose.** Language and type-system habits are what make C# read like C# rather than Java written in C# syntax.

6.4 · The lessons

#	Lesson	Phase	Hours
01	**The .NET Platform Map**	A · Foundations	4
02	**C# Language Essentials**	A · Foundations	6
03	**Types, OOP & Generics**	A · Foundations	6
04	**Collections, LINQ & Functional C#**	A · Foundations	5
05	**Async, Tasks & Concurrency**	B · Runtime & codebase	7
06	**VB.NET for Legacy Estates**	B · Runtime & codebase	5
07	**Solutions, MSBuild & Mono-repos**	B · Runtime & codebase	5
08	**ASP.NET Core Web APIs**	C · Services	7
09	**Data Access with EF Core**	C · Services	6
10	**Testing, TDD & 100% Coverage**	C · Services	6
11	**Security & Identity with Entra ID**	D · Production	6
12	**Cloud-Native .NET on AWS & Modernization**	D · Production	8
13	**Working in Visual Studio**	E · Tooling	5
	Total	• Estimate	**76**

6.5 · What carries over, what to learn, what to unlearn

Carries over unchanged

- OOP, SOLID and dependency-injection thinking
- Unit testing, TDD and coverage discipline
- Microservice, event-driven and cloud architecture
- Build-pipeline and mono-repo design instincts

Learn fresh

- Value types and struct copy semantics
- `async/await` and `Task` as the concurrency model
- LINQ and expression trees
- MSBuild evaluation and `Directory.Build.props`

Unlearn

- Type-erasure workarounds such as `Class<T>` tokens
- Checked exceptions and `throws` clauses
- Hand-written getters and setters — use properties
- "A newer runtime runs any older app" — not across a major version
- "The .exe is my app in native code" — it is a native launcher over IL

7 · Hands-on — build and run the lesson-01 samples

Three projects, two languages, one command each. `L01.PlatformKit` is a C# class library holding the `PlatformReport` record and the `PlatformProbe` that fills it. `L01.CSharpApp` and `L01.VbApp` are console apps that reference it and print what runtime they are on. Running both is the fastest way to see that the language is a front end and the runtime is shared.

Two repository-level files make every sample build the same way. `global.json` pins the SDK; `Directory.Build.props` sets the target framework, nullable reference types for C#, `Option Strict On` for VB, and treats warnings as errors so no sample teaches a habit the compiler is already complaining about.

7.1 · The two files every sample inherits

JSON — `global.json`

```json
{
  "sdk": {
    "version": "10.0.401",
    "rollForward": "latestFeature"
  }
}
```

Pins the SDK, not the runtime. `latestFeature` uses the highest installed 10.0 SDK at or above 10.0.401 — 10.0.500 beats 10.0.401 when both are installed — but never an 11.0 SDK. Use `feature` instead if you want the 10.0.4xx band preferred and later bands only as a fallback.

MSBuild / XML — `Directory.Build.props`

```xml
<Project>
  <!--
    Applies to EVERY sample project in this repository: MSBuild walks up from each project folder
    and imports the first Directory.Build.props it finds. Lesson 07 explains this file line by line.
  -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Deterministic>true</Deterministic>
    <!-- bin/ and obj/ go to one place instead of beside every project -->
    <UseArtifactsOutput>true</UseArtifactsOutput>
    <ArtifactsPath>$(MSBuildThisFileDirectory).build\artifacts</ArtifactsPath>
    <!-- a sample that warns is a sample that teaches the wrong habit -->
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <!-- ...but a newly published package advisory must not break an old lesson overnight -->
    <WarningsNotAsErrors>NU1901;NU1902;NU1903;NU1904</WarningsNotAsErrors>
  </PropertyGroup>

  <PropertyGroup Condition="'$(MSBuildProjectExtension)' == '.csproj'">
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <PropertyGroup Condition="'$(MSBuildProjectExtension)' == '.vbproj'">
    <OptionStrict>On</OptionStrict>
    <OptionExplicit>On</OptionExplicit>
    <OptionInfer>On</OptionInfer>
  </PropertyGroup>
</Project>
```

Read the conditions on the last two groups. `$(MSBuildProjectExtension)` lets one file give C# projects nullable reference types and VB projects `Option Strict On`. The `NU1901`–`NU1904` exclusion stops a newly published package advisory from failing an old lesson overnight.

7.2 · Commands

Terminal

```
# the SDK this repository pins (global.json) must be installed
dotnet --list-sdks

# from the repository root
dotnet run --project lesson-01-dotnet-platform-map/samples/L01.CSharpApp
dotnet run --project lesson-01-dotnet-platform-map/samples/L01.VbApp

# or build, test and run every sample of this lesson
python 0-script/verify_samples.py --only 1
```

> **There is no separate restore or build step:** `dotnet run` restores and builds first. `--project` takes a project folder or file. `dotnet --list-sdks` must show 10.0.401 or a later 10.0 SDK; if it shows none, see *Before you start*.

7.3 · What you should see

Output — C# app captured on the build machine

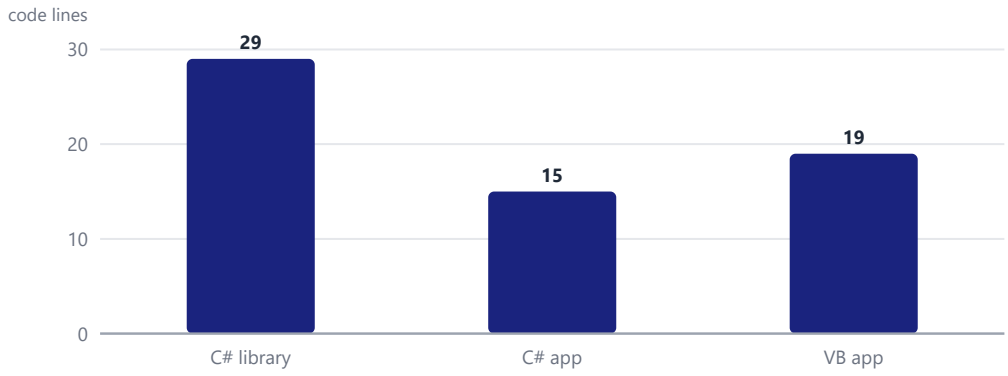
```
Hello from C# on .NET 10.0.12
  target    .NETCoreApp,Version=v10.0
  os        Microsoft Windows 10.0.26200
  arch      X64
  server GC False
  JIT on    True
  image     v4.0.30319
  assembly  L01.CSharpApp
  same?     False
  copy?     True
```

Output — VB app captured on the build machine

```
Hello from VB on .NET 10.0.12
  target    .NETCoreApp,Version=v10.0
  os        Microsoft Windows 10.0.26200
  arch      X64
  server GC False
  JIT on    True
  image     v4.0.30319
  assembly  L01.VbApp
  equal?    True
```

> **Your OS line, architecture and runtime patch number may differ; the other lines should not.** The first line is the runtime executing the app (10.0.12 here, newer than the 10.0.0 the app asked for); `target` is what it was built for. `JIT on  True` means a normal JIT-compiled run — publish with Native AOT and it prints False. `image  v4.0.30319` appears on .NET 10 exactly as it did on .NET Framework 4.

7.4 · Lines of code in the lesson-01 samples



non-blank, non-comment lines · measured from the sample files when this PDF was built

> **The VB app is 4 lines longer, and all of them are the `Module` / `Sub Main` wrapper and its `End` lines.** The rest is line-for-line, because VB continues a statement after an open parenthesis without a `_`. • **Measured** from the sample files at build time; line count is a readability signal, not a quality score.

⚠ Traps on day one

1. **"A newer runtime runs my older app."** A Java 8 JAR usually runs on a Java 21 JRE, but a `net8.0` app does not start on a host that has only .NET 10. The host rolls forward to a newer patch or minor, never a newer major, unless `RollForward` is `Major` — retarget instead (3.6).
2. **`ImageRuntimeVersion` says v4.0.30319.** That is the CLR version string written into the metadata header, frozen at CLR 4. For the target framework read `TargetFrameworkAttribute` (the `target` line in 7.3, or ILSpy), the `tfm` in `*.runtimeconfig.json`, or the project file. `RuntimeInformation.FrameworkDescription` tells you which runtime is executing the process, which can be newer.
3. **The .exe is not your program compiled to native code.** `L01.VbApp.exe` is a small native apphost that starts the runtime and loads the IL in `L01.VbApp.dll`. Copying only the .exe to another machine does not work.

4. **Reading a struct out of a list or a property gives you a copy.** Changing that copy leaves the original untouched, where changing a Java object through its reference would not — lesson 03 (Types, OOP & Generics).

✓ Checkpoint — you are ready for lesson 02 when

1. `python 0-script/verify_samples.py --only 1` reports 3/3 passed on your machine.
2. You can point at each box in diagram 2.1 and name its JVM counterpart.
3. Given a project file, you can say which row of table 3.4 it belongs to, what its support status is and which runtime it will start on.
4. You can explain why the VB app can compare the C# record but cannot copy it with `with`.

🔍 Check yourself

1. Why does an assembly built for .NET 10 report `ImageRuntimeVersion` v4.0.30319, and where do you read its target framework instead?
2. A repository has `packages.config`, `Global.asax` and `<TargetFrameworkVersion>v4.7.2`. Which runtime does it need, and why can it not run on a Linux container unchanged?
3. On what date do .NET 8 and .NET 9 leave support, and why is upgrading from 8 to 9 not a fix?
4. A host has only the .NET 10 runtime. Does a `net8.0` app start there, and what are your two ways out?
5. Which publish options would you combine for a latency-sensitive Lambda function, and what do you give up?
6. What can Visual Basic do with a C# `record`, and what can it not do?
7. With `rollForward: latestFeature` and version 10.0.401 in `global.json`, which SDK runs when both 10.0.401 and 10.0.500 are installed?

Lesson 01 in one line: .NET is a multi-language runtime shaped like the JVM — CoreCLR, IL assemblies, a tiered JIT and NuGet — shipped every November, with **.NET 10 LTS** as the target for new work and **10 Nov 2026** as the end of .NET 8 and 9. Apps roll forward within a major version, never across one. C# evolves; VB consumes.
Next: lesson 02 (C# Language Essentials) — the syntax and type-system habits that make C# read like C#.